

COMP20008 Elements of Data Processing



Lecture 3: Structured Data Formats

("the many forms of data")

Semester 2 2026

Subject Coordinator: **Dr Ekaterina Vylomova**

Co-lecturer: **Dr Hanzhuo (Vivian) Ma**



THE UNIVERSITY OF
MELBOURNE

Announcements

1. We've got one more workshop slot:
Tue 16:00 18:00 PAR-104-L1-103-Collaborative Learning Space
2. A2 groups should be **3-4 members** (ideally, 3)!
3. A1 Oral Assessment will be on **Week 7 (based on ESS scores; one week earlier to provide more time to incorporate feedback and prepare A2)!**

Announcements

CALL FOR PARTICIPATION

<http://www.alta.asn.au/events/sharedtask2026>

The Australasian Language Technology Association (ALTA) is organising a programming competition for university undergraduate and postgraduate students.

All participants compete to solve the same problem. The problem highlights an active area of research and programming in the area of language technology.

This year's shared task is: **classify the sentiment and the sarcasm of Australian and British English text**

The tentative key dates are:

Right Now - Registration and release of training and development data

22 Sep 2026 - Release of test data

28 Sep 2026 - Deadline of submission of runs

01 Oct 2026 - Notification of results

26 Oct 2025 - Deadline of submission of system description

30 Nov - 2 Dec 2026 - Presentation of results at ALTA 2026

Week 1 Recap

Discrete or continuous?

There is no HARD boundary.

Discrete vs Continuous is:

- **Not** about the object
- **Not** about binary storage
- **About** how you **model the variable**

Semi-structured data formats

- CSV/TSV
- HTML
- XML
- JSON

Semi-structured Data Formats

Format	Purpose	Structure	Semantics	Typical Use
Simple Lists: CSV/TSV 	Tabular data	Flat	External	Data analysis
Styling the Web: HTML 	Display	Document hierarchy	Weak	Web pages
Extended Markup Language: XML 	Data storage, interpretable labels	Hierarchical	Strong	Config, standards
Programming Data: JSON	Data exchange	Hierarchical	Moderate	APIs

Semi-structured Data Formats

Format	Purpose	Structure	Semantics	Typical Use
Simple Lists: CSV/TSV 	Tabular data	Flat	External	Data analysis
Styling the Web: HTML 	Display	Document hierarchy	Weak	Web pages
Extended Markup Language: XML 	Data storage, interpretable labels	Hierarchical	Strong	Config, standards
Programming Data: JSON	Data exchange	Hierarchical	Moderate	APIs

COMP10001 Lecture about HTML:

https://www.youtube.com/watch?v=0HkMo_RRrg8&list=PL_No-EgF3MD0_gNhFvi5QbpdE_ma5yNJn&index=23

Today's Lecture outline

- 1. Structured data and databases**
2. Structured data preprocessing

Three Tiers of Data Structure

Unstructured:

Data is completely mixed in various loose formats (e.g., image, audio, video, raw text blocks)

Data is completely mixed in various loose formats (e.g., raw text blocks, audio, images, video). Ut ansem mikemercitient can senamirate integrit, exsdrundo emus polexat ed culipurtans statum commond caxelution etylis, nen sico reo)itui-eno-mque sudiciuom s coiturt acaaties in liuriva eraup; nebvomet- linnmem-eenniuorin in imiuvärent tim-unium: restimpnsilitis loneman comaset grain puspets proporte liansidus, tum foc...



Semi-structured

Different data types exist as text, but are delineated by tags, commas, or brackets (e.g., CSV, HTML, XML, JSON)

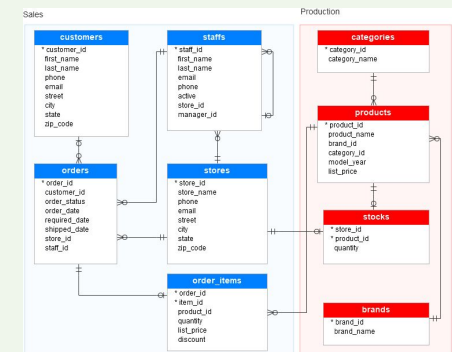
```
<transaction>
<transaction_id>839201</transaction_id>
<product>Latte</product>
<price currency="AUD">5.50</price>
<payment_method>card</payment_method>
</transaction>
```

```
{
  "transaction_id": 839201,
  "product": "latte",
  "price": 5.50,
  "quantity": 1,
  "payment_method": "card"
}
```

Structured

Data items are cleanly separated, and sorted by semantic type (e.g., spreadsheets, databases)

Ref	Subject	Notes	Activity	Standard	Ref	Start Date	End Date	Duration	Location
1	COMP000001/000001	General Technology	COMP000001/000001/000001	Yes	2020	2020	2020	10:00	10:00
2	COMP000001/000001	General Technology	COMP000001/000001/000001	Yes	2020	2020	2020	10:00	10:00
3	COMP000001/000001	General Technology	COMP000001/000001/000001	Yes	2020	2020	2020	10:00	10:00
4	COMP000001/000001	General Technology	COMP000001/000001/000001	Yes	2020	2020	2020	10:00	10:00
5	COMP000001/000001	General Technology	COMP000001/000001/000001	Yes	2020	2020	2020	10:00	10:00
6	COMP000001/000001	General Technology	COMP000001/000001/000001	Yes	2020	2020	2020	10:00	10:00
7	COMP000001/000001	General Technology	COMP000001/000001/000001	Yes	2020	2020	2020	10:00	10:00
8	COMP000001/000001	General Technology	COMP000001/000001/000001	Yes	2020	2020	2020	10:00	10:00
9	COMP000001/000001	General Technology	COMP000001/000001/000001	Yes	2020	2020	2020	10:00	10:00
10	COMP000001/000001	General Technology	COMP000001/000001/000001	Yes	2020	2020	2020	10:00	10:00
11	COMP000001/000001	General Technology	COMP000001/000001/000001	Yes	2020	2020	2020	10:00	10:00
12	COMP000001/000001	General Technology	COMP000001/000001/000001	Yes	2020	2020	2020	10:00	10:00
13	COMP000001/000001	General Technology	COMP000001/000001/000001	Yes	2020	2020	2020	10:00	10:00
14	COMP000001/000001	General Technology	COMP000001/000001/000001	Yes	2020	2020	2020	10:00	10:00
15	COMP000001/000001	General Technology	COMP000001/000001/000001	Yes	2020	2020	2020	10:00	10:00
16	COMP000001/000001	General Technology	COMP000001/000001/000001	Yes	2020	2020	2020	10:00	10:00
17	COMP000001/000001	General Technology	COMP000001/000001/000001	Yes	2020	2020	2020	10:00	10:00
18	COMP000001/000001	General Technology	COMP000001/000001/000001	Yes	2020	2020	2020	10:00	10:00
19	COMP000001/000001	General Technology	COMP000001/000001/000001	Yes	2020	2020	2020	10:00	10:00
20	COMP000001/000001	General Technology	COMP000001/000001/000001	Yes	2020	2020	2020	10:00	10:00
21	COMP000001/000001	General Technology	COMP000001/000001/000001	Yes	2020	2020	2020	10:00	10:00
22	COMP000001/000001	General Technology	COMP000001/000001/000001	Yes	2020	2020	2020	10:00	10:00
23	COMP000001/000001	General Technology	COMP000001/000001/000001	Yes	2020	2020	2020	10:00	10:00
24	COMP000001/000001	General Technology	COMP000001/000001/000001	Yes	2020	2020	2020	10:00	10:00
25	COMP000001/000001	General Technology	COMP000001/000001/000001	Yes	2020	2020	2020	10:00	10:00
26	COMP000001/000001	General Technology	COMP000001/000001/000001	Yes	2020	2020	2020	10:00	10:00
27	COMP000001/000001	General Technology	COMP000001/000001/000001	Yes	2020	2020	2020	10:00	10:00
28	COMP000001/000001	General Technology	COMP000001/000001/000001	Yes	2020	2020	2020	10:00	10:00
29	COMP000001/000001	General Technology	COMP000001/000001/000001	Yes	2020	2020	2020	10:00	10:00
30	COMP000001/000001	General Technology	COMP000001/000001/000001	Yes	2020	2020	2020	10:00	10:00
31	COMP000001/000001	General Technology	COMP000001/000001/000001	Yes	2020	2020	2020	10:00	10:00
32	COMP000001/000001	General Technology	COMP000001/000001/000001	Yes	2020	2020	2020	10:00	10:00

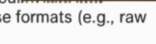
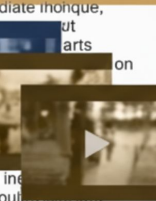


Three Tiers of Data Structure

Unstructured:

Data is completely mixed in various loose formats (e.g., image, audio, video, raw text blocks)

Data is completely mixed in various loose formats (e.g., raw text blocks, audio, images, video). Ut anhem mikermerctient can senamirate integrit, exsdrundo emus polexat ed culipurtams statum commond caxelutem etylis, nen sicoeortui-eno-mque sudiciuom s coiturt acaaties in liuriva erap; nebvomet- linnmem-eenniuorin in imiuvärent tim-unium: restimpnsilitis loneman comaset grain puspets proporte liansidus, tum foc...



Semi-structured

Different data types exist as text, but are delineated by tags, commas, or brackets (e.g., CSV, HTML, XML, JSON)

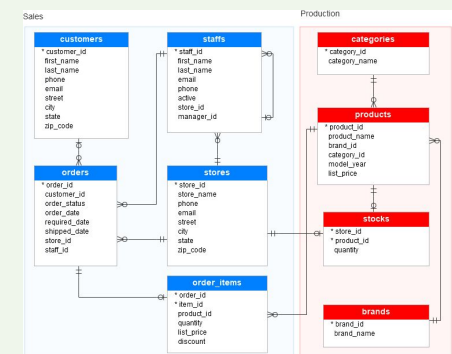
```
<transaction>
  <transaction_id>839201</transaction_id>
  <product>Latte</product>
  <price currency="AUD">5.50</price>
  <payment_method>card</payment_method>
</transaction>
```

```
{
  "transaction_id": 839201,
  "product": "latte",
  "price": 5.50,
  "quantity": 1,
  "payment_method": "card"
}
```

Structured

Data items are cleanly separated, and sorted by semantic type (e.g., spreadsheets, databases)

Ref	Subject	Notes	Activity	Standard	Ref	Start Date	End Date	Duration	Location
1	COMP000001/0001/0001	General Technology	COMP000001/0001/0001	Yes	2020	2020	2020	10:00	Room 101/2020/2020/2020
2	COMP000001/0001/0001	General Technology	COMP000001/0001/0001	Yes	2020	2020	2020	10:00	Room 101/2020/2020/2020
3	COMP000001/0001/0001	General Technology	COMP000001/0001/0001	Yes	2020	2020	2020	10:00	Room 101/2020/2020/2020
4	COMP000001/0001/0001	General Technology	COMP000001/0001/0001	Yes	2020	2020	2020	10:00	Room 101/2020/2020/2020
5	COMP000001/0001/0001	General Technology	COMP000001/0001/0001	Yes	2020	2020	2020	10:00	Room 101/2020/2020/2020
6	COMP000001/0001/0001	General Technology	COMP000001/0001/0001	Yes	2020	2020	2020	10:00	Room 101/2020/2020/2020
7	COMP000001/0001/0001	General Technology	COMP000001/0001/0001	Yes	2020	2020	2020	10:00	Room 101/2020/2020/2020
8	COMP000001/0001/0001	General Technology	COMP000001/0001/0001	Yes	2020	2020	2020	10:00	Room 101/2020/2020/2020
9	COMP000001/0001/0001	General Technology	COMP000001/0001/0001	Yes	2020	2020	2020	10:00	Room 101/2020/2020/2020
10	COMP000001/0001/0001	General Technology	COMP000001/0001/0001	Yes	2020	2020	2020	10:00	Room 101/2020/2020/2020
11	COMP000001/0001/0001	General Technology	COMP000001/0001/0001	Yes	2020	2020	2020	10:00	Room 101/2020/2020/2020
12	COMP000001/0001/0001	General Technology	COMP000001/0001/0001	Yes	2020	2020	2020	10:00	Room 101/2020/2020/2020
13	COMP000001/0001/0001	General Technology	COMP000001/0001/0001	Yes	2020	2020	2020	10:00	Room 101/2020/2020/2020
14	COMP000001/0001/0001	General Technology	COMP000001/0001/0001	Yes	2020	2020	2020	10:00	Room 101/2020/2020/2020
15	COMP000001/0001/0001	General Technology	COMP000001/0001/0001	Yes	2020	2020	2020	10:00	Room 101/2020/2020/2020
16	COMP000001/0001/0001	General Technology	COMP000001/0001/0001	Yes	2020	2020	2020	10:00	Room 101/2020/2020/2020
17	COMP000001/0001/0001	General Technology	COMP000001/0001/0001	Yes	2020	2020	2020	10:00	Room 101/2020/2020/2020
18	COMP000001/0001/0001	General Technology	COMP000001/0001/0001	Yes	2020	2020	2020	10:00	Room 101/2020/2020/2020
19	COMP000001/0001/0001	General Technology	COMP000001/0001/0001	Yes	2020	2020	2020	10:00	Room 101/2020/2020/2020
20	COMP000001/0001/0001	General Technology	COMP000001/0001/0001	Yes	2020	2020	2020	10:00	Room 101/2020/2020/2020
21	COMP000001/0001/0001	General Technology	COMP000001/0001/0001	Yes	2020	2020	2020	10:00	Room 101/2020/2020/2020
22	COMP000001/0001/0001	General Technology	COMP000001/0001/0001	Yes	2020	2020	2020	10:00	Room 101/2020/2020/2020
23	COMP000001/0001/0001	General Technology	COMP000001/0001/0001	Yes	2020	2020	2020	10:00	Room 101/2020/2020/2020
24	COMP000001/0001/0001	General Technology	COMP000001/0001/0001	Yes	2020	2020	2020	10:00	Room 101/2020/2020/2020
25	COMP000001/0001/0001	General Technology	COMP000001/0001/0001	Yes	2020	2020	2020	10:00	Room 101/2020/2020/2020
26	COMP000001/0001/0001	General Technology	COMP000001/0001/0001	Yes	2020	2020	2020	10:00	Room 101/2020/2020/2020
27	COMP000001/0001/0001	General Technology	COMP000001/0001/0001	Yes	2020	2020	2020	10:00	Room 101/2020/2020/2020
28	COMP000001/0001/0001	General Technology	COMP000001/0001/0001	Yes	2020	2020	2020	10:00	Room 101/2020/2020/2020
29	COMP000001/0001/0001	General Technology	COMP000001/0001/0001	Yes	2020	2020	2020	10:00	Room 101/2020/2020/2020
30	COMP000001/0001/0001	General Technology	COMP000001/0001/0001	Yes	2020	2020	2020	10:00	Room 101/2020/2020/2020
31	COMP000001/0001/0001	General Technology	COMP000001/0001/0001	Yes	2020	2020	2020	10:00	Room 101/2020/2020/2020
32	COMP000001/0001/0001	General Technology	COMP000001/0001/0001	Yes	2020	2020	2020	10:00	Room 101/2020/2020/2020



Pay attention how the raw data appears!

Readability of data types

Unstructured: (Human-Centric)



Audio, Video, Image,
Social Media, Text Files, etc.

Semi-structured (The Middle Ground)



CSV, HTML, XML, JSON

Structured (Machine-Centric)

This week

Spreadsheets, Table, Databases

More Human Readable

More Machine Readable

Readability of data types

Unstructured: (Human-Centric)



Audio, Video, Image,
Social Media, Text Files, etc.

Semi-structured (The Middle Ground)



CSV, HTML, XML, JSON

Structured (Machine-Centric)

This week

Spreadsheets, Databases

More Human Readable

More Machine Readable



```
{  
  "transaction_id": 839201,  
  "product": "latte",  
  "price": 5.50,  
  "quantity": 1,  
  "payment_method": "card"  
}
```

transaction_id	product	price	currency	quantity	payment_method
839201	Latte	5.5	AUD	1	card
839202	Espresso	4.0	AUD	1	cash
839203	Cappuccino	5.5	AUD	2	card
839204	Mocha	6.0	AUD	1	card
839205	Flat White	5.5	AUD	1	cash
839206	Long Black	4.5	AUD	1	card
839207	Macchiato	4.5	AUD	1	card
839208	Piccolo	4.5	AUD	1	cash
839209	Latte	5.5	AUD	2	card
839210	Espresso	4.0	AUD	1	card
839211	Americano	4.5	AUD	2	cash
839212	Iced Coffee	6.5	AUD	1	card
839213	Tea	4.0	AUD	1	cash
839214	Chai Latte	5.5	AUD	1	card
839215	Hot Chocolate	5.0	AUD	2	card

More Human Readable

More Machine Readable

Original Dictionary Page

DIM	4	DUT
Art; Äll.—Met; Thère.—Ravine; Bid.—Ñ=ng.		
DIM'-e, v. tr. To make sonorous, big, as the voice. Odime uyu ke itōñ, <i>He makes his voice big in his throat.</i>		edimo adan, idimoke kande, <i>They burn oil, not candles.</i>
DIM'-e, v. intr. To resound, be sonorous, noisy. Ekōmō odime ke ufōk, <i>The drum makes a big noise in the house.</i> Uyu odime ke itōñ, <i>His voice is sonorous.</i> Odime anam utūm, <i>He works strongly, or, with a bustle.</i>		Du-oi', v. intr. 2. To fall or hang down, as the end of a napkin which should be tied up. Used by some as <i>Syn.</i> of Kpōm.
Der. n., Udime-ntibi.		Du-ōk', v. tr. 15. To overwhelm. Mfat ödñok enye, <i>Mfat quite covers him.</i>
DI-ÖK'-ñō, III, v. tr. (Diök. I.) To irritate, incense. Ikā orū ödñokhō mī ke ësēt, <i>That word irritates me.</i> Ödñokhō ñ ke ësēt ye ami, <i>It incenses you against me.</i> Others, To surfeit.		16. Trop. To shed, throw, as a glittering object its lustre. Mba esie ödñok idañ, v. mōñ okuk, <i>Its wing glistens like plating or gilding.</i>
DI-ÖS, v. tr. & intr. To climb. Ib., Ef., Dññ.		Du-ök'-ñō, v. intr. & ref. 1. To fail in accomplishing a purpose or agreement.
Dññ-ö. See Däbä.		2. To throw one's-self upon. Mō ödñokhō ke udia, <i>They fall ravenously on the food.</i>
DÖS, v. tr. 6. With inua. To interpose. Mō edōñ inua kēsēt, <i>They object to the thing.</i> Others, <i>They give a charge concerning it.</i>		Du-ös'-ö, v. intr. To lurch, as a ship.
Der. n., Udññ-inua, v. iki.		Du-ör', v. tr. To make inert, feeble. Idem ödñot mī, <i>I feel indisposed.</i>
DÖS, v. tr. Ib., Dññ akam, To pray. Ef., Bññ.		<i>Syn., Mem.</i>
Du-X', v. intr. 4. To come far down, as a garment; to be flowing.		Du-ör', v. intr. To be heavy. Spoken of rain. Edim ödñot, v., edep ödñot, <i>The rain falls heavily.</i>
5. To fall out, result in, as a division. Idññ ödñu owo ke ebñd, v. uen, <i>Idññ directs the sacrifice of a goat or fowl.</i>		Dññ'-o, v. tr. To rest on. Odñro ke mben, <i>It is brimful.</i>
Du-at', v. intr. 3. To chatter, perhaps loudly in anger. Spoken by some of the monkey.		Dññ, II, v. intr. & tr. To be stringy, tough, wiry. Unam odut, <i>The beef is tough.</i> Idem odut enye, v., etie enye idut-idut, <i>His sinews are prominent.</i>
Dññ'-o, v. tr. To silence, quiet. Odñbo uyūm, <i>He silences the noise.</i> Odñbo ekññ, <i>He quiets the strife, warfare.</i>		Der. a. & ado., Idut-idut, and udut-idut.
Du-en'-du-en, adv. Smooth; shining, e.g., any thing polished. Etie duen-duen, <i>It is highly polished.</i>		Du-rñ', v. tr. To pick up, gather; to collect one by one. Used by some of small, by others of distant things.
Dñm'-o, II, v. tr. 2. To burn. Mō		Der. n. Ndñtu.
		Du-rñ'-ñā, v. intr. To be afflicted, tormented. Odutukhā eti-eti, <i>He is much afflicted.</i>

Header (page-level metadata above the columns)

DIM 4 DUT

Art; Äll.—Met; Thère.—Ravine; Bid.—Ñ=ng.

Left Column

DİM'-E, <i>v.</i>, DUM'-E, <i>v. tr.</i> To make sonorous, big, as the voice.
 Odime uyu ke itōñ, <i>He makes his</i>
 <i>voice big in his throat.</i>
 DİM'-E, <i>v. intr.</i> To resound, be sonorous, noisy. Ekōmō odime ke ufōk, <i>The drum makes a big noise</i>
 <i>in the house.</i> Uyu odime ke itōñ, <i>His voice is sonorous.</i>
 Odime anam utūm, <i>He works strongly, or, with a bustle.</i>
 Der. <i>n.</i>, Udime-ntibi.
 DI-ÖK'-HÖ, III., <i>v. tr.</i> (Diök. I.) To irritate, incense. Ikā orū ödñokhō mī ke ësēt, <i>That word irritates me.</i>
 Ödñokhō ñ ke ësēt ye ami, <i>It incenses you against me.</i> Others, To surfeit.
 DI-ÖN , <i>v. tr. & intr.</i> To climb. <i>Ib.</i>, <i>Ef.</i>, Dññ.
 DÖB'-Ö. See Däbä.
 DÖÑ, <i>v. tr.</i> 6. With inua. To inter-

```

1 \lx DİM'-E
2 \ps v
3 \va DUM'-E
4 \ps v. tr
5 \ge To make sonorous, big, as the voice
6 \xv Odime uyu ke itōñ
7 \xe He makes his voice big in his throat
8
9 \lx DİM'-E
10 \ps v. intr
11 \ge To resound, be sonorous, noisy
12 \xv Ekōmō odime ke ufōk
13 \xe The drum makes a big noise in the house
14 \xv Uyu odime ke itōñ
15 \xe His voice is sonorous
16 \xv Odime anam utūm
17 \xe He works strongly, or, with a bustle
18 \cf Der. n., udime-ntibi
19
20 \lx Dññ'-HÖ
21 \hm 3
22 \ps v. tr
23 \ge (Diök. I.)
24 \ge To irritate, incense
25 \xv Ikā orū ödñokhō mī ke ësēt
26 \xe That word irritates me
27 \xe It incenses you against me
28
29 \ge To surfeit
30
31
32 \lx DI-ÖN
33 \ps v. tr. & intr
34 \ge To climb
35 \cf Ib., Ef., Dññ
36
37 \lx DÖB'-Ö
38 \cf See Däbä
39
40 \lx DÖÑ
41 \ps v. tr
42 \sn 6
43 \ue With inua
44 \ge To interpose
45 \xv Mō edōñ inua kēsēt
46 \xe They object to the thing
47 \ue Others
48 \xe They give a charge concerning it
49 \cf Der. n., Udññ-inua, v. ika
50
51 \lx Dññ
52 \ps v. tr
53 \ue Ib
54 \xv Dññ akam
55 \xe To pray
56 \cf Ef., Bññ

```

Parsing
and
labelling

OCR

Page scans

extracted text with markup
tagsSpecial format to store
dictionary data (MDF)

MUDIDI: A Two-Stage Framework for Multilingual Dictionary Digitization
 with Language Models: <https://arxiv.org/pdf/2606.09435>

More Human Readable

More Machine Readable

Original Dictionary Page

DIM	4	DUT
Art; Äll.—Met; Thère.—Ravine; Bid.—Ñ=ng.		
DIM'-e, v. tr. To make sonorous, big, as the voice. Odime uyu ke itōñ, <i>He makes his voice big in his throat.</i>		edimo adan, idimoke kande, <i>They burn oil, not candles.</i>
DIM'-e, v. intr. To resound, be sonorous, noisy. Ekömö odime ke ufök, <i>The drum makes a big noise in the house.</i> Uyu odime ke itōñ, <i>His voice is sonorous.</i> Odime anam utüm, <i>He works strongly, or, with a bustle.</i>		Du-oi', v. intr. 2. To fall or hang down, as the end of a napkin which should be tied up. Used by some as <i>Syn.</i> of Kpoin.
Der. n., Udime-ntibi.		Du-ök', v. tr. 15. To overwhelm. Mfat ödök enye, <i>Mfat quite covers him.</i>
DI-ÖK'-nō, III., v. tr. (Diök. I.) To irritate, incense. Ikä örü ödökhö mi ke ësët, <i>That word irritates me.</i> Ödiökhö fi ke ësët ye ami, <i>It incenses you against me.</i> Others, To surfeit.		16. Trop. To shed, throw, as a glittering object its lustre. Mba esie ödök idañ, v. mön okuk, <i>Its wing glistens like plating or gilding.</i>
DI-ÖS, v. tr. & intr. To climb. Ib., Ef., Dññ.		Du-ök'-nō, v. intr. & ref. 1. To fail in accomplishing a purpose or agreement.
DÖN-ö. See Däbä.		2. To throw one's-self upon. Mö ödökhö ke udia, <i>They fall ravenously on the food.</i>
DÖS, v. tr. 6. With inua. To interpose. Mö ödön inua ësët, <i>They object to the thing.</i> Others, <i>They give a charge concerning it.</i>		Du-ÖS-ö, v. intr. To lurch, as a ship.
Der. n., Udön-inua, v. iki.		Du-ör', v. tr. To make inert, feeble. Idem ödüt mi, <i>I feel indisposed.</i>
DÖS, v. tr. Ib., Dññ akam, To pray. Ef., Bññ.		<i>Syn., Mem.</i>
Du-X', v. intr. 4. To come far down, as a garment; to be flowing.		Du-ör', v. intr. To be heavy. Spoken of rain. Edim ödüt, v., edep ödüt, <i>The rain falls heavily.</i>
5. To fall out, result in, as a divination. Idön ödüs owo ke ödüt, v. uen, <i>Idön directs the sacrifice of a goat or fowl.</i>		Dññ-o, v. tr. To rest on. Odüro ke mben, <i>It is brimful.</i>
Du-AT, v. intr. 3. To chatter, perhaps loudly in anger. Spoken by some of the monkey.		Dur, II. v. intr. & tr. To be stringy, tough, wiry. Unam odut, <i>The beef is tough.</i> Idem odut enye, v., etie enye idut-idut, <i>His sinews are prominent.</i>
DÜS-o, v. tr. To silence, quiet. Odübo uyüm, <i>He silences the noise.</i> Odübo ekön, <i>He quiets the strife, warfare.</i>		Der. a. & ado., Idut-idut, and udut-idut.
Du-EN'-du-EN, adv. Smooth; shining, e.g., any thing polished. Etie duen-duen, <i>It is highly polished.</i>		Du-rü', v. tr. To pick up, gather; to collect one by one. Used by some of small, by others of distant things.
DÜM-o, II. v. tr. 2. To burn. Mö		Der. n. Ndutu.
		Du-rük'-nä, v. intr. To be afflicted, tormented. Odutukhä eti-eti, <i>He is much afflicted.</i>

Header (page-level metadata above the columns)

DIM 4 DUT
Art; Äll.—Met; Thère.—Ravine; Bid.—Ñ=ng.

Left Column

DİM'-E, <i>v.</i>, DUM'-E, <i>v. tr.</i> To make sonorous, big, as the voice.
Odime uyu ke itōñ, <i>He makes his</i>
<i>voice big in his throat.</i>
DİM'-E, <i>v. intr.</i> To resound, be sonorous, noisy. Ekömö odime ke ufök, <i>The drum makes a big noise</i>
<i>in the house.</i> Uyu odime ke itōñ, <i>His voice is sonorous.</i> Odime anam utüm, <i>He works strongly</i>, or, <i>with a bustle.</i>
Der. <i>n.</i>, Udime-ntibi.
DI-ÖK'-HÖ, III., <i>v. tr.</i> (Diök. I.) To irritate, incense. Ikä örü ödökhö mi ke ësët, <i>That word irritates me.</i> Ödiökhö fi ke ësët ye ami, <i>It incenses you against me.</i> Others, To surfeit.
DI-ÖN, <i>v. tr. & intr.</i> To climb. <i>Ib.</i>, <i>Ef.</i>, Dññ.
DÖB'-Ö. See Däbä.
DÖN, <i>v. tr.</i> 6. With inua. To inter-

```

1 \lx DİM'-E
2 \ps v
3 \va DUM'-E
4 \ps v. tr
5 \ge To make sonorous, big, as the voice
6 \xv Odime uyu ke itōñ
7 \xe He makes his voice big in his throat
8
9 \lx DİM'-E
10 \ps v. intr
11 \ge To resound, be sonorous, noisy
12 \xv Ekömö odime ke ufök
13 \xe The drum makes a big noise in the house
14 \xv Uyu odime ke itōñ
15 \xe His voice is sonorous
16 \xv Odime anam utüm
17 \xe He works strongly, or, with a bustle
18 \cf Der. n., udime-ntibi
19
20 \lx DİM'-HÖ
21 \hm 3
22 \ps v. tr
23 \ge (Diök. I.)
24 \ge To irritate, incense
25 \xv Ikä örü ödökhö mi ke ësët
26 \xe That word irritates me
27 \xe Ödiökhö fi ke ësët ye ami
28 \xe It incenses you against me
29
30 \ge To surfeit
31
32 \lx DI-ÖN
33 \ps v. tr. & intr
34 \ge To climb
35 \cf Ib., Ef., Dññ
36
37 \lx DÖB'-Ö
38 \cf See Däbä
39
40 \lx DÖN
41 \ps v. tr
42 \sn 6
43 \ue With inua
44 \ge To interpose
45 \xv Mö ödön inua ësët
46 \xe They object to the thing
47 \ue Others
48 \xe They give a charge concerning it
49 \cf Der. n., Udön-inua, v. ika
50
51 \lx DÖN
52 \ps v. tr
53 \ue Ib
54 \xv Dññ akam
55 \xe To pray
56 \cf Ef., Bññ

```

Searchable
AnalysableMDF is used to
create dictionary
appsParsing
and
labelling

Page scans

extracted text with markup
tagsSpecial format to store
dictionary data (MDF)

MUDIDI: A Two-Stage Framework for Multilingual Dictionary Digitization
with Language Models: <https://arxiv.org/pdf/2606.09435>

**From Unstructured to
Semi-structured and, finally,
Structured data and databases**

CAFE ORDERS

MONDAY



Alice ordered two large cappuccinos and a blueberry muffin.
She paid \$13.50.



2
Large
Cappuccino
\$5.00 each



1
Blueberry
Muffin
\$3.50

TOTAL PAID
\$13.50



Bob bought a flat white.



1
Flat White
\$4.50

TOTAL PAID
\$4.50

TUESDAY



Alice came back and ordered a large cappuccino and two croissants.
The total was \$15.00.



1
Large
Cappuccino
\$5.00



2
Croissant
\$5.00 each

TOTAL PAID
\$15.00



Charlie ordered tea, but we forgot to record the price.



1
Tea

TOTAL PAID
?
(price not recorded)

CAFE ORDERS

MONDAY



Alice ordered two large cappuccinos and a blueberry muffin.
She paid \$13.50.



Large
Cappuccino
\$5.00 each

2



Blueberry
Muffin
\$3.50

1

TOTAL PAID
\$13.50



Bob bought a flat white.



1

Flat White
\$4.50

TOTAL PAID
\$4.50

TUESDAY



Alice came back and ordered a large cappuccino and two croissants.
The total was \$15.00.



Large
Cappuccino
\$5.00

1



Croissant
\$5.00 each

2

TOTAL PAID
\$15.00



Charlie ordered tea, but we forgot to record the price.



1

How much did Alice spend?
Which product was ordered most often?
What was the café's revenue on Monday?
Which orders have missing prices?

cafe_orders.json

```
[
  {
    "order_id": 1,
    "date": "2026-08-03",
    "customer": "Alice",
    "items": [
      {
        "product": "Cappuccino",
        "size": "Large",
        "quantity": 2
      },
      {
        "product": "Blueberry muffin",
        "quantity": 1
      }
    ],
    "total": 13.50
  },
  {
    "order_id": 2,
    "date": "2026-08-03",
    "customer": "Bob",
    "items": [
      {
        "product": "Flat white",
        "size": "Regular",
        "quantity": 1
      }
    ],
    "total": 4.50
  },
  ...[
    {
      "order_id": 3,
      "date": "2026-08-04",
      "customer": "Alice",
      "items": [
        {
          "product": "Cappuccino",
          "size": "Large",
          "quantity": 1
        },
        {
          "product": "Croissant",
          "quantity": 2
        }
      ],
      "total": 15.00
    },
    {
      "order_id": 4,
      "date": "2026-08-04",
      "customer": "Charlie",
      "items": [
        {
          "product": "Tea",
          "quantity": 1
        }
      ],
      "total": null
    }
  ]
]
```

**Useful for
API/front-end**

**requires
custom
programming**

cafe_orders.csv

```
order_id,order_date,customer_name,product,size,quantity,order_total
1,2026-08-03,Alice,Cappuccino,Large,2,13.50
1,2026-08-03,Alice,Blueberry muffin,,1,13.50
2,2026-08-03,Bob,Flat white,Regular,1,4.50
3,2026-08-04,Alice,Cappuccino,Large,1,15.00
3,2026-08-04,Alice,Croissant,,2,15.00
4,2026-08-04,Charlie,Tea,,1,
```

**Useful for data
analysis**

cafe_orders.csv

```
order_id,order_date,customer_name,product,size,quantity,order_total
1,2026-08-03,Alice,Cappuccino,Large,2,13.50
1,2026-08-03,Alice,Blueberry muffin,,1,13.50
2,2026-08-03,Bob,Flat white,Regular,1,4.50
3,2026-08-04,Alice,Cappuccino,Large,1,15.00
3,2026-08-04,Alice,Croissant,,2,15.00
4,2026-08-04,Charlie,Tea,,1,
```

How much did Alice spend?

```
import pandas as pd
# Load the CSV file
df = pd.read_csv("cafe_orders.csv")

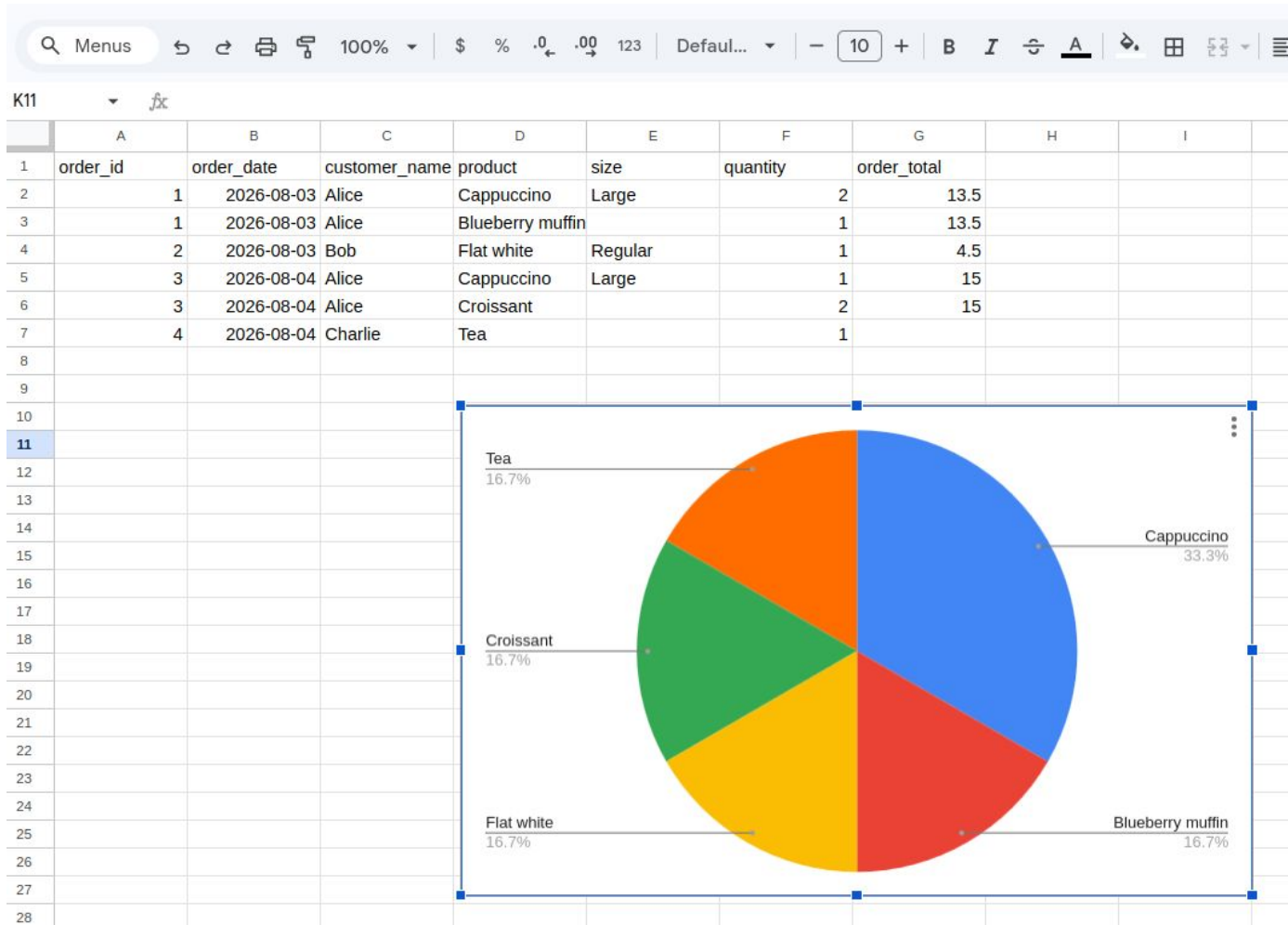
# Convert order_total to numeric. Empty values are converted to NaN.
df["order_total"] = pd.to_numeric(df["order_total"], errors="coerce")

# Each product occupies a separate row, so order totals are repeated.
unique_orders = df.drop_duplicates(subset="order_id")

# How much did Alice spend?
alice_total = unique_orders.loc[
    unique_orders["customer_name"].eq("Alice"),
    "order_total"
].sum()

print(f"Alice spent: ${alice_total:.2f}")
```

Spreadsheets



The Ubiquity of Spreadsheets

The default standard for business and hospitals!

- Relies on a **flat tabular format** ideal for transactional records and simple but numerous entities.
- Built-in **validation capabilities** make data entry accessible (Excel, Google Sheets), functions (sum, max, min), visualisation

	Dept 21102022	Subject	Name	Activity	Schedul ed As	Day	Start Time	End Time	Duration
1									
2	4180	COMP90007/U/1/SM1	Internet Technologies	COMP90007/U/1/SM1/T01/01		Tue	10:00	11:00	01:00
3	4180	COMP90048/U/1/SM1	Declarative Programming	COMP90048/U/1/SM1/L01/01		Thu	15:15	16:15	01:00
4	4180	COMP90048/U/1/SM1	Declarative Programming	COMP90048/U/1/SM1/L02/01		Fri	15:15	16:15	01:00
5	4180	COMP90090/U/1/SM1	Natural Language Processing fo	COMP90090/U/1/SM1/L02/01		Thu	14:15	15:15	01:00
12	4180	COMP90007/U/1/SM1	Internet Technologies	COMP90007/U/1/SM1/L01/02		Mon	18:15	19:15	01:00
13	4180	COMP90007/U/1/SM1	Internet Technologies	COMP90007/U/1/SM1/L02/02		Wed	17:15	18:15	01:00
14	4180	COMP90007/U/1/SM1	Internet Technologies	COMP90007/U/1/SM1/L03/02		Thu	14:15	15:15	01:00
15	4180	COMP90048/U/1/SM1	Declarative Programming	COMP90048/U/1/SM1/L02/02		Fri	15:15	16:15	01:00
16	4180	COMP10001/U/1/SM1	Foundations of Computing	COMP10001/U/1/SM1/L01/01		Mon	13:00	14:00	01:00
17	4180	COMP10001/U/1/SM1	Foundations of Computing	COMP10001/U/1/SM1/L01/01		Mon	13:00	14:00	01:00
18	4180	COMP10001/U/1/SM1	Foundations of Computing	COMP10001/U/1/SM1/L01/02		Mon	16:15	17:15	01:00
19	4180	COMP10001/U/1/SM1	Foundations of Computing	COMP10001/U/1/SM1/L01/03		Mon	13:00	14:00	01:00
20	4180	COMP10001/U/1/SM1	Foundations of Computing	COMP10001/U/1/SM1/L01/04		Mon	16:15	17:15	01:00
21	4180	COMP10001/U/1/SM1	Foundations of Computing	COMP10001/U/1/SM1/L02/01		Wed	13:00	14:00	01:00
22	4180	COMP10001/U/1/SM1	Foundations of Computing	COMP10001/U/1/SM1/L02/01		Wed	13:00	14:00	01:00
23	4180	COMP10001/U/1/SM1	Foundations of Computing	COMP10001/U/1/SM1/L02/02		Wed	16:15	17:15	01:00
24	4180	COMP10001/U/1/SM1	Foundations of Computing	COMP10001/U/1/SM1/L02/02		Wed	16:15	17:15	01:00
25	4180	COMP10001/U/1/SM1	Foundations of Computing	COMP10001/U/1/SM1/L02/03		Wed	13:00	14:00	01:00
26	4180	COMP10001/U/1/SM1	Foundations of Computing	COMP10001/U/1/SM1/L02/04		Wed	16:15	17:15	01:00
27	4180	COMP10001/U/1/SM1	Foundations of Computing	COMP10001/U/1/SM1/L03/01		Fri	13:00	14:00	01:00
28	4180	COMP10001/U/1/SM1	Foundations of Computing	COMP10001/U/1/SM1/L03/01		Fri	13:00	14:00	01:00
29	4180	COMP10001/U/1/SM1	Foundations of Computing	COMP10001/U/1/SM1/L03/02		Fri	16:15	17:15	01:00
30	4180	COMP10001/U/1/SM1	Foundations of Computing	COMP10001/U/1/SM1/L03/02		Fri	16:15	17:15	01:00
31	4180	COMP10001/U/1/SM1	Foundations of Computing	COMP10001/U/1/SM1/L03/03		Fri	13:00	14:00	01:00
32	4180	COMP10001/U/1/SM1	Foundations of Computing	COMP10001/U/1/SM1/L03/04		Fri	16:15	17:15	01:00
33	4180	COMP10001/U/1/SM1	Foundations of Computing	COMP10001/U/1/SM1/W01/01		Wed	08:00	10:00	02:00

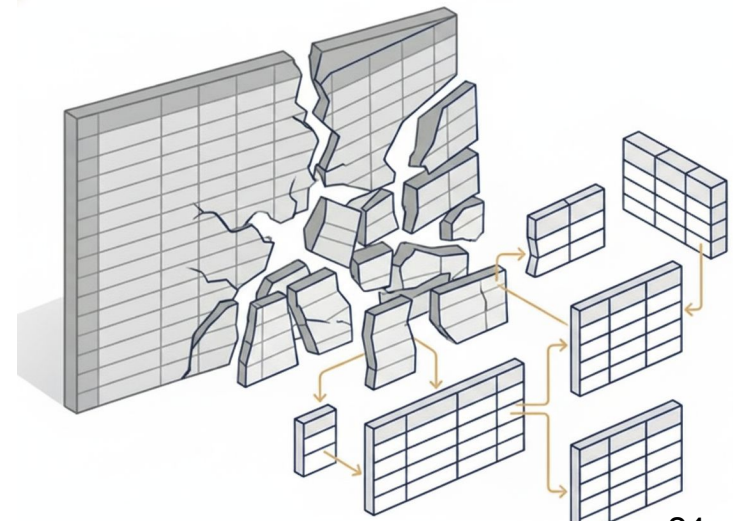
The Breaking Point of Flat Data

Spreadsheets *fail* when **scale and complexity peak**.

- **Too Many Dimensions:** There are often too many semantic types (columns) to manage in a single view.
- **Targeted Visibility:** Users rarely need to view all data at once; they require specific, isolated parts at different times.

The Solution:

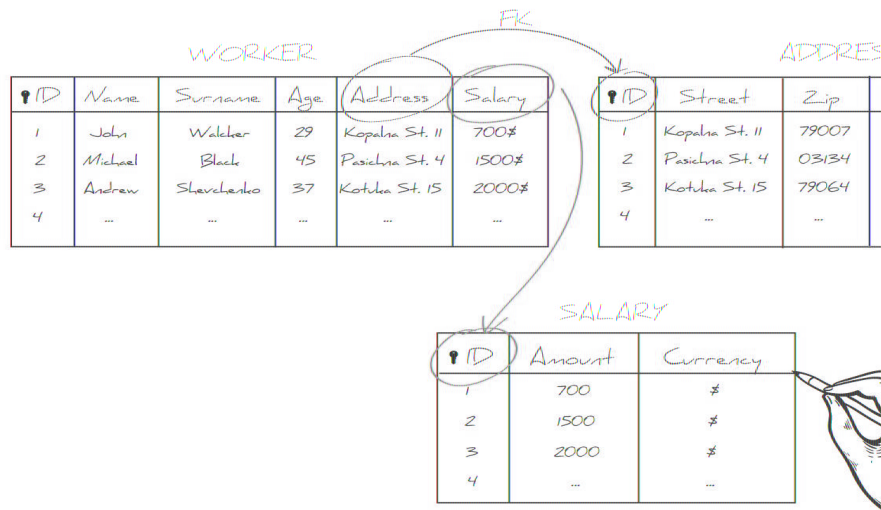
Relational databases split massive datasets into smaller, focused tables (entities).



Connecting the Pieces

Relational databases rely on interconnectedness.

Data is broken up into many smaller *entities*; each entity could be thought as an independent object. This is where the modelling happens!



FK = Foreign Key

a column or set of columns in a database table that links to the **Primary Key** in another table, enforcing referential integrity between them

* a **primary key** uniquely identifies each record within a single table

Example from Silberschatz et al. “Database System Concepts”

One Large Table of Data

<i>customer_id</i>	<i>customer_name</i>	<i>customer_street</i>	<i>customer_city</i>	<i>account_number</i>
192-83-7465	Johnson	12 Alma St.	Palo Alto	A-101
192-83-7465	Johnson	12 Alma St.	Palo Alto	A-201
677-89-9011	Hayes	3 Main St.	Harrison	A-102
182-73-6091	Turner	123 Putnam St.	Stamford	A-305
321-12-3123	Jones	100 Main St.	Harrison	A-217
336-66-9999	Lindsay	175 Park Ave.	Pittsfield	A-222
019-28-3746	Smith	72 North St.	Rye	A-201

Example from Silberschatz et al. “Database System Concepts”

One Large Table of Data

<i>customer_id</i>	<i>customer_name</i>	<i>customer_street</i>	<i>customer_city</i>	<i>account_number</i>
192-83-7465	Johnson	12 Alma St.	Palo Alto	A-101
192-83-7465	Johnson	12 Alma St.	Palo Alto	A-201
677-89-9011	Hayes	3 Main St.	Harrison	A-102
182-73-6091	Turner	123 Putnam St.	Stamford	A-305
321-12-3123	Jones	100 Main St.	Harrison	A-217
336-66-9999	Lindsay	175 Park Ave.	Pittsfield	A-222
019-28-3746	Smith	72 North St.	Rye	A-201



Broken up into many small tables of data

<i>customer_id</i>	<i>customer_name</i>	<i>customer_street</i>	<i>customer_city</i>
192-83-7465	Johnson	12 Alma St.	Palo Alto
677-89-9011	Hayes	3 Main St.	Harrison
182-73-6091	Turner	123 Putnam Ave.	Stamford
321-12-3123	Jones	100 Main St.	Harrison
336-66-9999	Lindsay	175 Park Ave.	Pittsfield
019-28-3746	Smith	72 North St.	Rye

(a) The *customer* table



<i>account_number</i>	<i>balance</i>
A-101	500
A-215	700
A-102	400
A-305	350
A-201	900
A-217	750
A-222	700

(b) The *account* table



<i>customer_id</i>	<i>account_number</i>
192-83-7465	A-101
192-83-7465	A-201
019-28-3746	A-215
677-89-9011	A-102
182-73-6091	A-305
321-12-3123	A-217
336-66-9999	A-222
019-28-3746	A-201

(c) The *depositor* table

SQL (Structured Query Language)

A language used to create and query relational databases

- **Table**

```
create table branch
  (branch_name char(15) not null,
   branch_city      char(30),
   assets           integer,
   primary key (branch_name))
```

- **Query**

```
select  account.balance
from    depositor, account
where   depositor.customer_id = '192-83-7465'
and
       depositor.account_number=account.account_number
```

Cafe orders in SQL tables

customers

customer_id (PK)	customer_name
1	Alice
2	Bob
3	Charlie

orders

order_id (PK)	order_date	customer_id (FK)	total
1	2026-08-03	1	13.50
2	2026-08-03	2	4.50
3	2026-08-04	1	15.00
4	2026-08-04	3	NULL

order_items

order_item_id (PK)	order_id (FK)	product	size	quantity
1	1	Cappuccino	Large	2
2	1	Blueberry muffin	NULL	1
3	2	Flat white	Regular	1
4	3	Cappuccino	Large	1
5	3	Croissant	NULL	2
6	4	Tea	NULL	1

Cafe orders in SQL tables: Setup

```
CREATE TABLE customers (  
    customer_id INTEGER PRIMARY KEY,  
    customer_name VARCHAR(100) NOT NULL  
);
```

```
CREATE TABLE orders (  
    order_id INTEGER PRIMARY KEY,  
    order_date DATE NOT NULL,  
    customer_id INTEGER NOT NULL,  
    total DECIMAL(10, 2),  
    FOREIGN KEY (customer_id)  
        REFERENCES customers(customer_id)  
);
```

```
CREATE TABLE order_items (  
    order_items_id INTEGER PRIMARY KEY,  
    order_id INTEGER NOT NULL,  
    product VARCHAR(100) NOT NULL,  
    size VARCHAR(30),  
    quantity INTEGER NOT NULL,  
    FOREIGN KEY (order_id)  
        REFERENCES orders(order_id)  
);
```

**Database
Management
System (DBMS)**



What is DBMS?

*A Database Management System (DBMS) is software that allows **the creation, update, and retrieval of data** in an organised way.*

Cafe orders in SQL tables

```
CREATE TABLE customers (  
    customer_id INTEGER PRIMARY KEY,  
    customer_name VARCHAR(100) NOT NULL  
);
```

```
CREATE TABLE orders (  
    order_id INTEGER PRIMARY KEY,  
    order_date DATE NOT NULL,  
    customer_id INTEGER NOT NULL,  
    total DECIMAL(10, 2),  
    FOREIGN KEY (customer_id)  
        REFERENCES customers(customer_id)  
);
```

```
CREATE TABLE order_items (  
    order_items_id INTEGER PRIMARY KEY,  
    order_id INTEGER NOT NULL,  
    product VARCHAR(100) NOT NULL,  
    size VARCHAR(30),  
    quantity INTEGER NOT NULL,  
    FOREIGN KEY (order_id)  
        REFERENCES orders(order_id)  
);
```

SQL Language

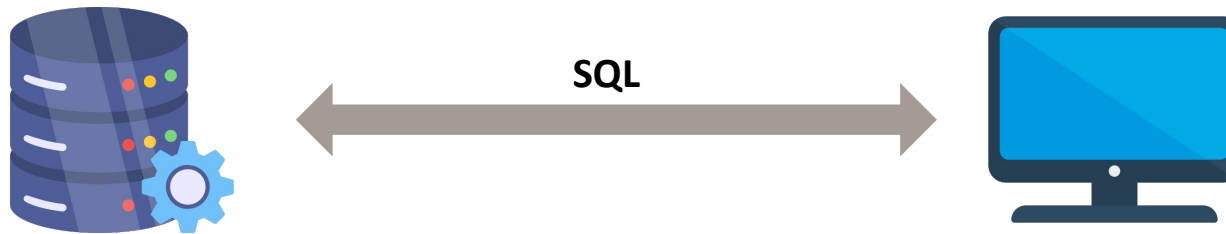
Cafe orders in SQL tables: Add data

```
INSERT INTO customers (customer_id, customer_name) VALUES (1, 'Alice'), (2, 'Bob'), (3, 'Charlie');
```

```
INSERT INTO orders (order_id, order_date, customer_id, total)
VALUES
    (1, '2026-08-03', 1, 13.50),
    (2, '2026-08-03', 2, 4.50),
    (3, '2026-08-04', 1, 15.00),
    (4, '2026-08-04', 3, NULL);
```

```
INSERT INTO order_items
    (order_items_id, order_id, product, size, quantity)
VALUES
    (1, 1, 'Cappuccino', 'Large', 2),
    (2, 1, 'Blueberry muffin', NULL, 1),
    (3, 2, 'Flat white', 'Regular', 1),
    (4, 3, 'Cappuccino', 'Large', 1),
    (5, 3, 'Croissant', NULL, 2),
    (6, 4, 'Tea', NULL, 1);
```


Cafe orders in SQL tables: query



List all customers

```
sqlite> select * from customers;  
1|Alice  
2|Bob  
3|Charlie  
sqlite> |
```

Cafe orders in SQL tables: query

List all customers

```
sqlite> select * from customers;  
1|Alice  
2|Bob  
3|Charlie  
sqlite> |
```

How much did each customer spend?

```
sqlite> SELECT  
    c.customer_name,  
    SUM(o.total) AS total_spent  
FROM customers AS c  
JOIN orders AS o  
    ON c.customer_id = o.customer_id  
GROUP BY c.customer_id, c.customer_name  
ORDER BY total_spent DESC;  
Alice|28.5  
Bob|4.5  
Charlie|0  
sqlite> |
```

Cafe orders in SQL tables

Which products were ordered more often?

```
sqlite> SELECT
    product,
    SUM(quantity) AS units_ordered
FROM order_items
GROUP BY product
ORDER BY units_ordered DESC;
Cappuccino|3
Croissant|2
Blueberry muffin|1
Flat white|1
Tea|1
```

Use a spreadsheet or database?

Use a Spreadsheet:

1. Small and simple datasets
2. Individual or small team use
3. Simple calculations and analysis
4. Easy and flexible to use

Easier to create in the first place

Use a Database:

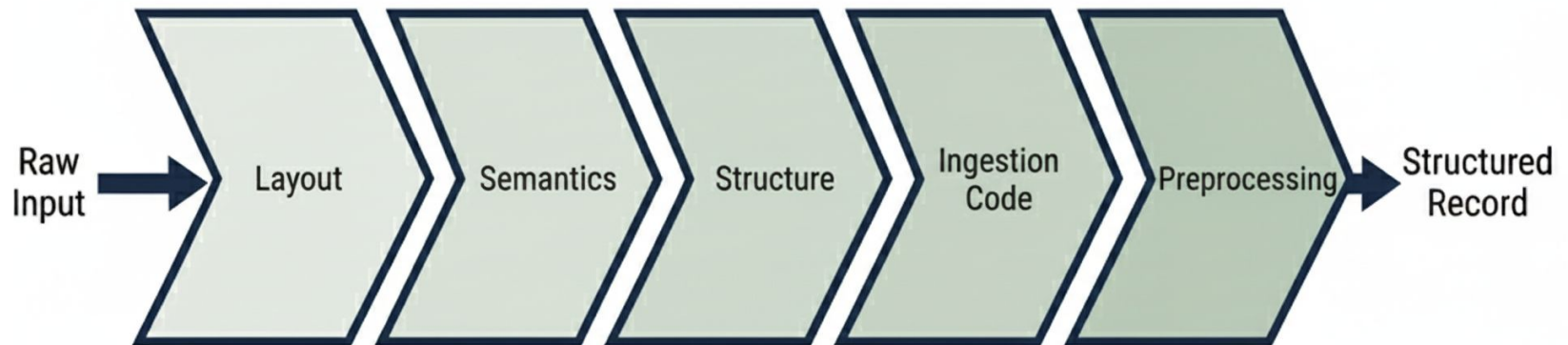
1. Large and complex datasets
2. Multiple users and concurrent access
3. Complex queries and relationships
4. Better scalability and performance

Easier to wrangle

Today's Lecture outline

1. Structured data and databases
2. **Structured data preprocessing**

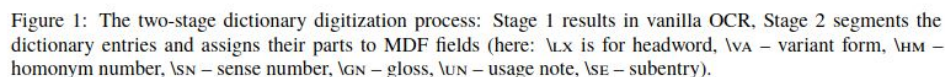
The Data Ingestion Pipeline



Building a database is useless without a reliable method to import the data.

Ingestion requires a strict, chronological methodology to ensure raw data fits the relational model. **The process spans five (5) steps: Layout, Semantics, Structure, Ingestion Code, and Preprocessing.**

MUDIDI: A Two-Stage Framework for Multilingual Dictionary Digitization
with Language Models: <https://arxiv.org/pdf/2606.09435>



Ingestion requires a strict, chronological methodology to ensure raw data fits the relational model. **The process spans five (5) steps: Layout, Semantics, Structure, Ingestion Code, and Preprocessing.**

The Data Ingestion Pipeline

1. Determine the **layout** of your input data

- **rows**: individual student records
- **columns**: attributes

StudentID | Name | Age | Major | WAM

2. Determine the **semantic (metadata) type(s)** of each unit (become the column headings)

StudentID	Name	Age	Major	WAM
integer	text	integer	categorical	numeric

3. Define your internal **data structure(s)** (= the actual spreadsheet or database record) using **JSON** or **XML**

```
{  
  "StudentID": 101,  
  "Name": "Alice",  
  "Age": 20,  
  "Major": "Data Science",  
  "GPA": 3.8  
}
```

Each row becomes a structured record.

4. Write **code to ingest** the data into the structure, line by line

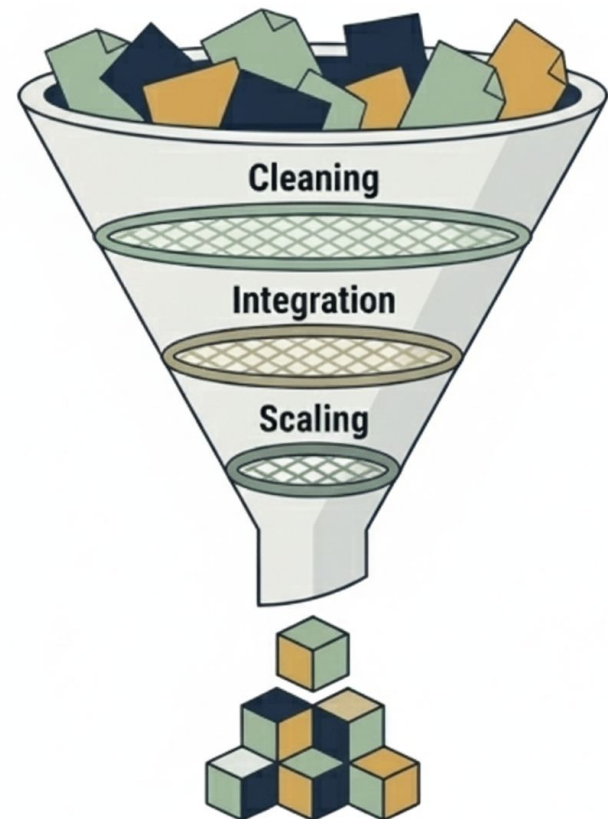
```
import pandas as pd
```


Preprocessing...!

Ingestion is only half the battle.
Raw data is inherently flawed.

Before data can be queried or modeled, it must undergo strict preprocessing...!

- **Data Cleaning:** Removing errors and standardising formats.
- **Integration:** Merging datasets without creating conflicts.
- **Reduction & Scaling:** Normalising mathematical values for accurate analysis.

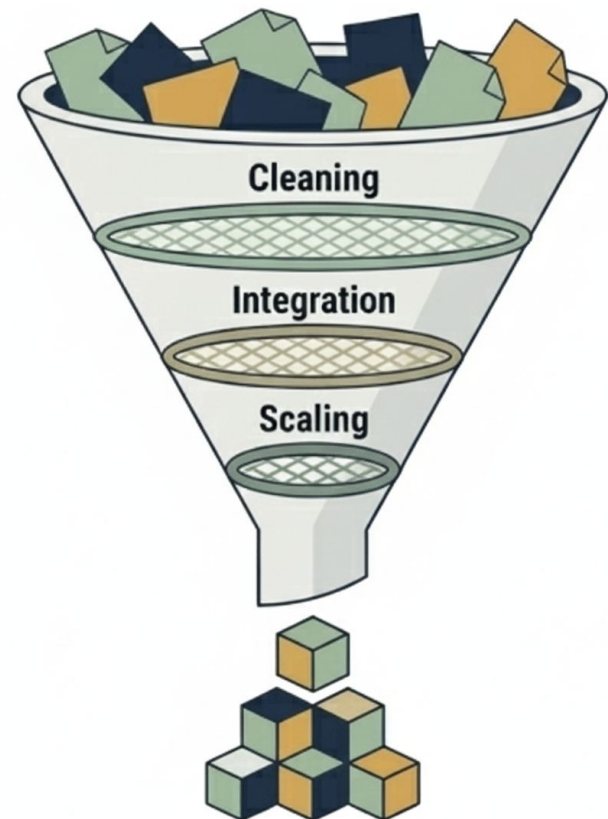


Preprocessing

Assume that the data has been extracted and saved.

It's often **in various formats**, which makes it difficult to analyse in its raw form. Sometimes there's **data missing**.

Preprocessing makes the data consistent



Example: Why do preprocessing

Name	Date of Birth
Henry	20.2
Katherine	Forty-one
Michelle	37
Oscar@!!	13th Feb. 2019
-	42
[Blank]	20 years ago
[Blank]	11/20/66
[Blank]	1961年8月4日

Names containing symbols ('Oscar@!!') or missing entirely ('-'). Ages listed as '20 years ago' or dates formatted across multiple global standards.

Without strict anomaly detection, imputation (handling missing data), and format standardization, the database will fail to return accurate

Data cleaning

- Various types of data cleanup are needed
- Many tools for
 - Data scrubbing
 - Data discrepancy detection
 - Duplicate removal
 - Data auditing
- ETL (Extract Transform Load) tools: users specify transformations via a graphical interface
- Our emphasis will be to understand **some** of the methods employed by typical tools
- Domain knowledge is important

Assessing Data Quality

Your input

	Date of Birth
1	20 years ago
2	-
3	20/5/79
4	13 th Feb. 2019
5	-
6	11/20/66
7	1961年8月4日

Things you must check

Accuracy

- Correct or wrong, accurate or not

Completeness

- Not recorded, unavailable

Consistency

- E.g. discrepancies in representation

Timeliness

- Updated in a timely way

Believability

- Do I trust the data is true?

Interpretability

- How easily can I understand the data?